I) Introduction

React permet la création d'interfaces utilisateurs avec un outil rapide et modulaire. La construction d'une application React ce fait globalement en créant des composants.

React ne manipule pas directement le DOM du navigateur. À la place, React génère un DOM virtuel. Au moment venu, il réconcilie ce DOM virtuel avec le DOM du navigateur.

Le DOM (Document Object Model), est généré par votre navigateur depuis le HTML pour afficher une page.

Pour manipuler les objets React, les équipes de React ont créé le JSX, leur propre syntaxe basée sur JavaScript, qui permet de mêler HTML et JS. Cela permet d'utiliser notre syntaxe sous forme de tags directement dans le code JavaScript.

La syntaxe JSX demande à suivre quelques règles :

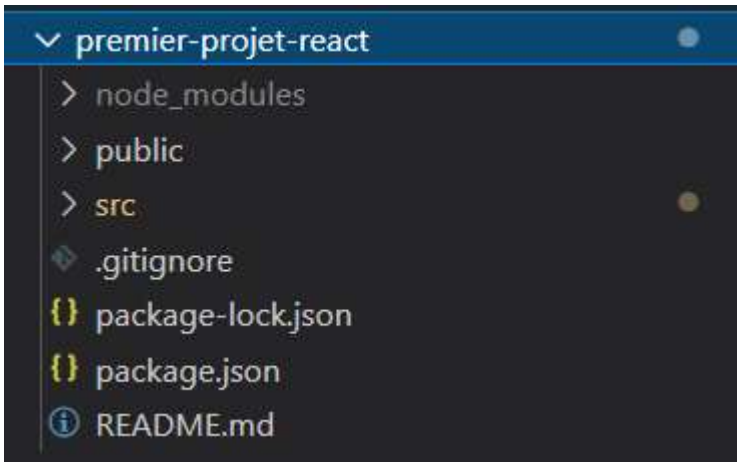
- **Deux composants ou plus doivent toujours être entouré d'un élément parent**
- **Les noms des composants doivent être en majuscules**
- **Les balises des composants doivent être refermées**

Les équipes de Facebook ont créé un outil permettant de construire une application complète React.

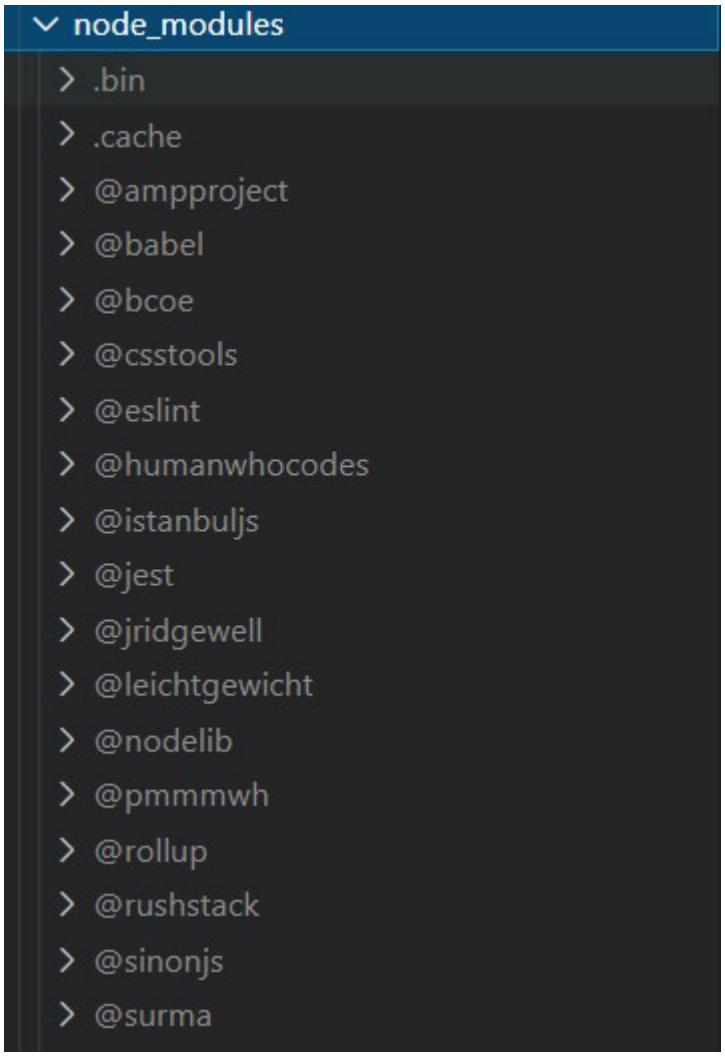
Create React App dispose de plusieurs outils déjà préconfigurés qui vont apporter de nombreuses fonctionnalités comme :

- **Gérer les différentes bibliothèques de notre application**
- **Optimiser le chargement de notre code dans le navigateur**
- **Faciliter l'expérience de développement, en rechargeant la page lors de la modification du code**

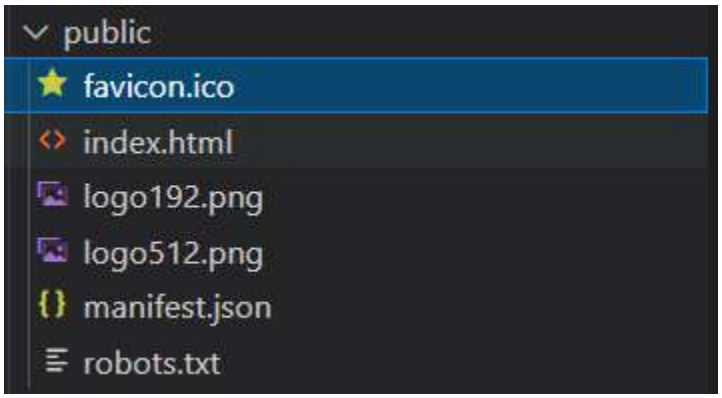
Lors de la création d'un projet CRA, une multitude de fichiers ce sont créés :



Le dossier node_modules comporte toutes les bibliothèques que notre application pourra utiliser.

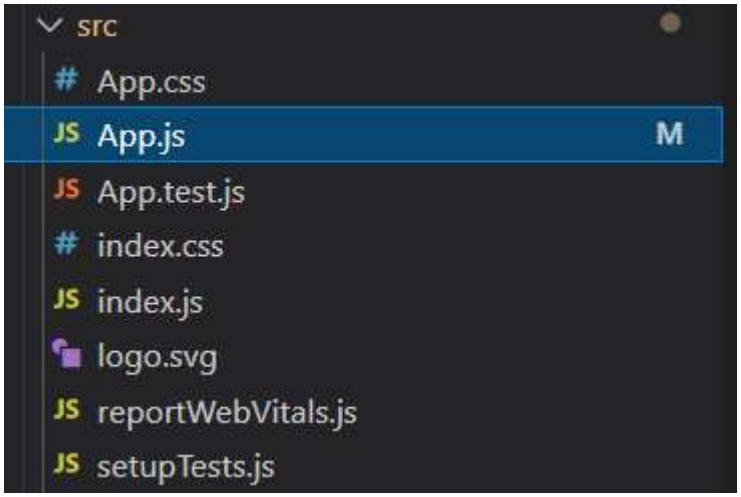


Un dossier public où se trouve notre fichier index.html qui est tout simplement notre template de l'application ainsi que d'autres fichiers.

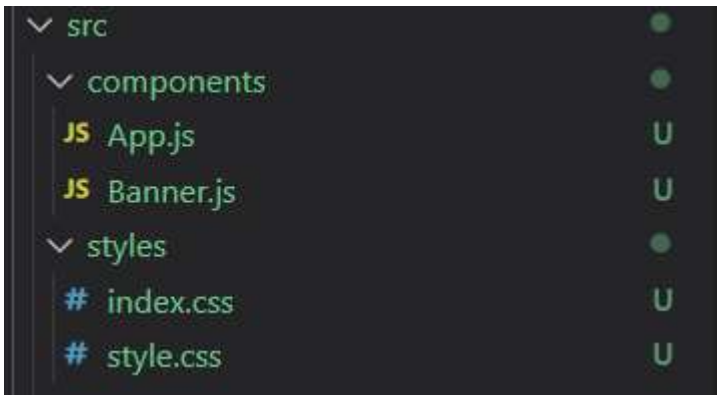


Enfin, un dossier src qui comporte des fichiers Javascript notamment le composant principal de l'application dans le fichier App.js.

Ce dossier permettra d'y ajouter nos propres composants, nos propres styles CSS...



Afin de mieux regrouper nos composants et nos pages de style CSS, on peut créer un dossier composants et un dossier styles.



II) Les composants

1. Création d'un composant

Un composant regroupe à la fois le HTML, le JS et le CSS. Il va pouvoir être réutilisable pour permettre de construire notre interface utilisateur.

Pour créer un composant React, on va créer un fichier JS qui contiendra une fonction qui va retourner un élément React.

```
function Banner()
{
    return <h1>La maison Jungle</h1>
}

export default Banner
```

Pour pouvoir l'afficher dans notre application, on va l'importer dans notre fichier App.js et l'ajouter dans le composant App.

```
import Banner from './Banner'

function App() {

  return <Banner/>
}

export default App;
```

La maison Jungle

2) Styliser un composant

Maintenant pour pouvoir styliser un élément, il suffit de sélectionner notre élément dans un fichier CSS grâce à un attribut, ici className et non pas class comme en HTML classique puis de lui ajouter du style.

```
import '../styles/style.css';

function Banner()
{
    return <h1 className="banner-title">La maison Jungle</h1>

}

export default Banner
```

```
.banner-title
{
    color: black;
    text-align: right;
    padding: 32px;
    border-bottom: solid 3px black;
}
```

La maison Jungle

---

3) Création d'une liste de composants dynamiques

Il est possible de créer une liste de composants dynamiques.

J'ai créé un fichier JS qui comporte une liste de plantes avec pour chacune sa catégorie, son nom et son id.

On l'importe dans notre composant qui va regrouper nos plantes.

Enfin, on utilise la méthode map() qui permet d'itérer sur un tableau de données et de créer notre liste de composants.

```
export const plantList = [
    {
        name: 'monstera',
        category: 'classique',
        id: '1ed'
    },
    {
        name: 'ficus lyrata',
        category: 'classique',
        id: '2ab'
    },
    {
        name: 'pothos argenté',
        category: 'classique',
        id: '3sd'
    },
    {
        name: 'yucca',
        category: 'classique',
        id: '4kk'
    },
    {
        name: 'olivier',
        category: 'extérieur',
        id: '5pl'
```

```
import { plantList } from "../datas/plantList";

function ShoppingList() {

    return (
        <div>
            <ul className="shoppingList-plantes">
                {plantList.map((plant) => (
                    <li key={plant.id} className="shoppingList-item">{plant.name}</li>
                ))}
            </ul>
        </div>

    )
}

export default ShoppingList
```

On peut également afficher des éléments en fonction d'une condition dans nos composants.

4) Conditions à l'intérieur des composants

Il est tout à fait possible de mettre une condition dans un composants afin qu'il affiche une valeur plus appropriée en fonction de la condition.

J'ai ajouté à plusieurs plantes dans ma liste de plantes (montré précédemment) une propriété 'isSpecialOffer', un booléen qui va permettre de savoir si elle est en promotion.

Enfin, on réalise une condition ici ternaire sur chaque plante pour savoir si elle est en promotion et si c'est le cas on ajoute le mot 'Solde'.

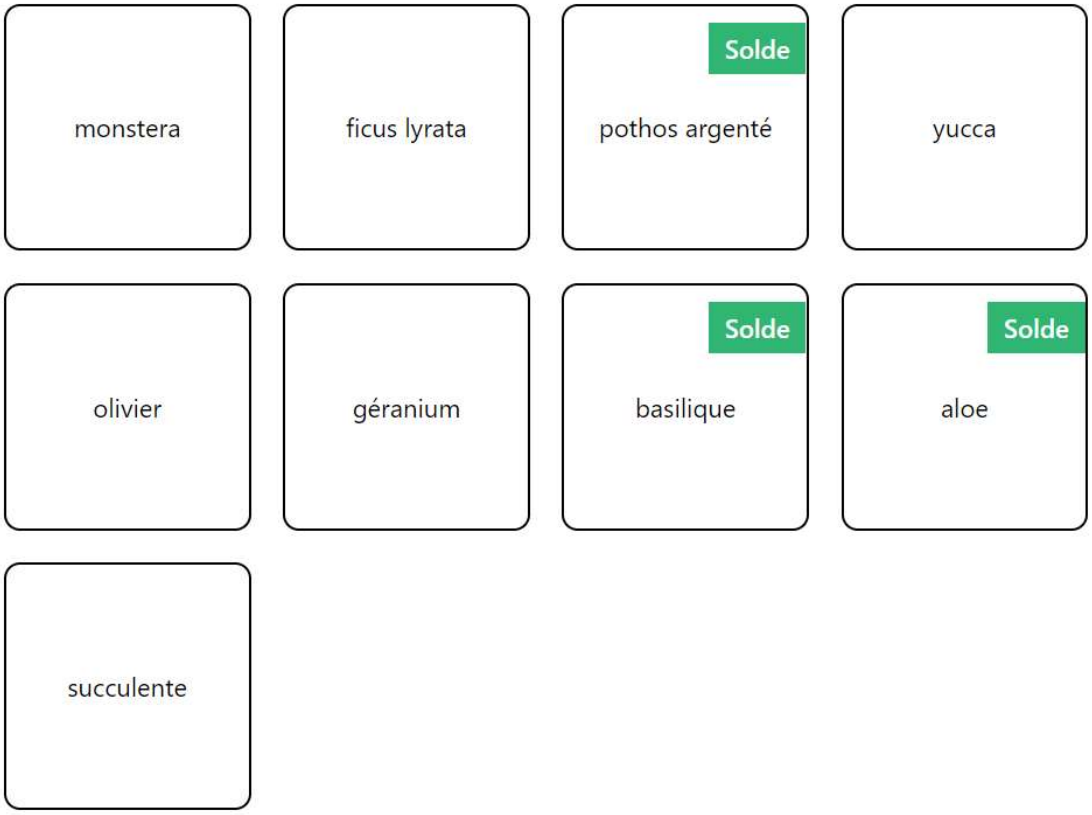
```
import { plantList } from "../datas/plantList";

function ShoppingList() {

    return (
        <div>
            <ul className="shoppingList-plantes">
                {plantList.map((plant) => (
                    <li key={plant.id} className="shoppingList-item">
                    {plant.name}
                    {plant.isSpecialOffer ? <div className="shoppingList-item-promo">Solde</div> : null}
                    </li>
                ))}
            </ul>
        </div>


    )
}

export default ShoppingList
```



III) Les props

Les props sont des objets que l'on peut récupérer dans les paramètres de notre composant fonction.

Afin de faire évoluer notre application de plantes, on peut ajouter une propriété ‘light’ and ‘water’ pour chaque plante dans notre liste qui vont signifier le besoin d’ensoleillement et d’arrosage dont la plante a besoin.

On va donc créer un composant ‘CareScale’ qui prendra en paramètre le type de besoin et le niveau de besoin. On récupère par la suite nos paramètres qui vont pouvoir être traité.

```
import Sun from '../assets/sun.svg'
import Water from '../assets/water.svg'

function CareScale({careType, scaleValue}) {


    const range = [1, 2, 3]

    const scaleType = careType === 'light' ? ( <img src={Sun} alt='sun-icon' /> ) : ( <img src={Water} alt='water-icon' /> )

    return (

        <div>
            {range.map((rangeElem) =>
                scaleValue >= rangeElem ? <span key={rangeElem.toString()}>{scaleType}</span> : null
            )}
        </div>
    )
}

export default CareScale
```

La syntaxe utilisée pour les paramètres de cet composant fonction s’appelle : **la déstructuration**

Elle permet d’assigner une ou plusieurs valeur(s) de propriété d’un objet à une variable ou comme dans cet exemple directement dans les paramètres. Cela permet d’éviter une multiplication de déclaration de variable et d’avoir un code plus compact.

```
import CareScale from "./CareScale";
import '../styles/style.css';


function PlantItem({id, name, cover, light, water})
{
    return (
        <div key={id} className="plantItem-plante">
            <img src={cover} className="plantItem-plante-img" alt=""></img>
            <h2>{name}</h2>
            <div>
                <CareScale careType='water' scaleValue={water}/>
                <CareScale careType='light' scaleValue={light}/>
            </div>
        </div>
    )
}

export default PlantItem
```

Ici, le composant PlantItem sert à afficher les éléments pour chaque plante.

On retrouve en props de cette fonction les informations nécessaires pour permettre l'affichage de chaque plante.

Mais aussi l'utilisation du composant CareScale étant un composant enfant du composant PlantItem où on lui passe les props **careType** et **scaleValue**.

Puis le composant PlantItem qui est aussi un composant enfant de ShoppingList.

```
import { plantList } from "../datas/plantList";
import PlantItem from "./PlantItem";

function ShoppingList() {

    return (
        <div>
            <ul className="shoppingList-plantes">
                {plantList.map((plant) => (
                    <PlantItem id={plant.id} name={plant.name} cover={plant.cover} light={plant.light} water={plant.water}/>
                ))}
            </ul>
        </div>


    )
}

export default ShoppingList
```

Monstera

Ficus Lyrata

Pothos Argenté

Yucca

Olivier

Géranium

Basilique

Aloe

Succulente

IV) La gestion d’évènements de composant

En React, on va parler d’évènements synthétiques.

React passe par défaut un objet en paramètre aux fonctions indiquées en callback des évènements.

```
<input placeholder='Entrez votre mail ' onChange={handleInput} value={inputValue}/>
```

```
function handleInput(event)
{
    console.log(event);
}
```

```
SyntheticBaseEvent {_reactName: 'onChange', _targetInst: n
▼ull, type: 'change', nativeEvent: InputEvent, target:
 input, …} ℹ
   bubbles: true
   cancelable: false
   currentTarget: null
   defaultPrevented: false
   eventPhase: 3
 ▶ isDefaultPrevented: ƒ functionThatReturnsFalse()
 ▶ isPropagationStopped: ƒ functionThatReturnsFalse()
   isTrusted: true
 ▶ nativeEvent: InputEvent {isTrusted: true, data: null, isC
 ▼ target: input
     value: "test"
```

Cet objet d’événement comprend de nombreuse propriété dont la valeur de l’action sur l’élément émis par l’utilisateur.

Il est possible de pouvoir contrôlé un formulaire grâce au state local.

Le state local nous permet de garder des informations. Ces informations sont spécifiques à un composant et elles proviennent d'une interaction que l'utilisateur a eue avec le composant.

```
const [inputValue, setInputValue] = useState('')
```

Cette ligne de code permet de déclarer l'état initial pour ma variable inputValue et la fonction setInputValue pour la modifier ainsi que de pouvoir préciser une valeur ou non par défaut.

```
function Footer() {
    const [inputValue, setInputValue] = useState('')

    function handleInput(event)
    {
        setInputValue(event.target.value)
    }

    function handleClickButton()
    {

        if(!inputValue.includes('@'))
        {
            alert('L\'adresse mail n\'est pas valide, il manque un @');
        }
        else
        {
            alert('L\'adresse mail est valide');
        }
    }

    return (
        <footer className='footer'>
            <div className='footer-elem'>
                Pour les passionnés de plantes 🌿🌵🌵
            </div>
            <div className='footer-elem'>Laissez-nous votre mail :</div>
            <input type={"text"} placeholder='Entrez votre mail ' onChange={handleInput} value={inputValue}/>
            <button className='footer-btn' onClick={handleClickButton} >Tester</button>
        </footer>
    )
}

export default Footer
```

Ici, on peut voir que j'appelle la fonction handleInput lors de l'événement onChange sur mon input. Cela permet à chaque changement de la valeur de mon input de modifier ma variable d'état local.

Grâce à l'événement onClick sur mon bouton qui appelle une fonction qui va permettre de vérifier ma variable d'état local pour savoir si c'est bien une adresse mail et d'afficher une alert en fonction du cas.

Les avantages de ces contrôles sont de permettre d'interagir directement avec l'utilisateur en récupérant sa ou ses données mais aussi de pouvoir filtrer grâce à des conditions mises dessus.